arXiv:2312.04556v2 [cs.CL] 2 Apr 2024

Large Language Models for Mathematicians

Simon Frieder*, Julius Berner†, Philipp Petersen‡, Thomas Lukasiewicz§

Abstract

Large language models (LLMs) such as ChatGPT have received immense interest for their general-purpose language understanding and, in particular, their ability to generate high-quality text or computer code. For many professions, LLMs represent an invaluable tool that can speed up and improve the quality of work. In this note, we discuss to what extent they can aid professional mathematicians. We first provide a mathematical description of the transformer model used in all modern language models. Based on recent studies, we then outline best practices and potential issues and report on the mathematical abilities of language models. Finally, we shed light on the potential of LLMs to change how mathematicians work.

1 Introduction

Large language models (LLMs) recently revolutionized the field of natural language processing, bringing forth unparalleled progress for applications such as conversational bots or text generation. Advances in deep-learning algorithms, computational resources, as well as substantial engineering efforts, allowed to train language models with billions of parameters and on massive datasets, such as the 800GB large *The Pile* [14]. Being trained on increasingly larger datasets, language models infer essential aspects of the structure of language[1], paving the way from task-specific to general-purpose language understanding. As such, these models can also parse mathematical questions and solve (sometimes surprisingly complex) mathematical problems and tasks—however, they also make embarrassing mistakes.

Let us have a look at two examples that ChatGPT (web version of 30-January-2023) produced[2]. Both examples below were taken from [13]; the input to the model, as well as its output, consists of LaTeX code, which we have compiled below for better readability:

"prompt": "Find a function $f:\mathbb{R}\to\mathbb{R}$ that is continuous at precisely one point.",
"output": "One example of such a function is

$$f(x)=\begin{cases} x^2 & \text{if } x\neq 0\\ 1 & \text{if } x=0.\end{cases}$$

This function is continuous at $x=0$ but not at any other point."

"prompt": "Let μ be a measure, $\|f\|_\infty=\inf\{M:\mu(\{x:|f(x)|>M\})=0\}$ and $\|g\|_1=\int|g|d\mu$. Prove that $\int|fg|d\mu\leq\|g\|_1\|f\|_\infty$.",
"output": "Let $M=\|f\|_\infty$ and $\epsilon>0$. Then, by the definition of $\|f\|_\infty$, we have $\mu(\{x:|f(x)|>M+\epsilon\})=0$. Therefore,

$$\int|fg|d\mu=\int_{|f|\leq M+\epsilon}|fg|d\mu+\int_{|f|>M+\epsilon}|fg|d\mu\leq(M+\epsilon)\int|g|d\mu+\int_{|f|>M+\epsilon}|f||g|d\mu.$$

The second term on the right side is zero because $\mu(x:|f(x)|>M+\epsilon)=0$. So we have $\int|fg|d\mu\leq(M+\epsilon)\int|g|d\mu=(M+\epsilon)\|g\|_1$. As ϵ can be arbitrarily small, we can let it go to zero and we get $\int|fg|d\mu\leq M\|g\|_1=\|f\|_\infty\|g\|_1$. Hence proved."

---

*Department of Computer Science, University of Oxford, Oxford, UK
†Department of Computing and Mathematical Sciences, Caltech, Pasadena, US
‡Faculty of Mathematics and Research Network Data Science, University of Vienna, Vienna, Austria
§Institute of Logic and Computation, Vienna University of Technology, Vienna, Austria, and Department of Computer Science, University of Oxford, Oxford, UK

[1]We note that language models work best in English since the most extensive datasets, including those that contain mathematics, are available in English.

[2]The first example can by now be correctly answered by the GPT-4 model [26], which was since released. We explain the acronym "GPT" in Section 2.

The first response is nonsensical on all levels, whereas the second is correct and helpful. With these examples in mind, we might ask ourselves:

How can large language models assist human mathematicians in their work?

To address this question, this article proceeds in the following way: First, we provide an overview of modern language models. We clarify the theory of how language models work, how their main building block—the *transformer* architecture—is set up, and why these models can perform sufficient mathematics to assist mathematicians in their daily work. Understanding this architecture will also highlight how an LLM produces an answer to a mathematical question – which differs tremendously from how a mathematician arrives at an answer. Then, we present empirical evidence attesting to the abilities of language models, in particular state-of-the-art models, such as ChatGPT and GPT-4. We end with an outlook on the potential future impacts on mathematicians and mathematics in general.

2 Overview of Modern Language Models

The concept of language models has a long history. One of the pioneering achievements, dating back to the year 2000, presented one of the initial instances of what we now refer to as *word embeddings* within the framework of neural networks [3]; see Section 3 for a definition.

Most previous approaches were rooted in estimating probabilities over *trigrams* (or, more general, n-grams). An n-gram is a sequence of n adjacent elements from a string of word pieces, so-called *tokens*, which could be syllables, letters, words, or base pairs according to the context. In the sentence "`The quick brown fox jumps over the lazy dog`", the sequence "`quick brown fox`" is an example of a trigram. Models based on n-grams had severe limitations: For example, if a trigram does not appear in the training corpus (or contains words that were not in the vocabulary of the corpus), no meaningful way of estimating its probability existed. By using a form of word embeddings, these problems are circumvented. The model proposed by [3] dominated all other pure n-gram models. The authors note that improvements can be made regarding the "*architecture, computational efficiency, and taking advantage of prior knowledge*".

The introduction of the *transformer architecture* [40] in 2017 marked the most striking advancement in terms of neural network architectures: On the one hand, the attention mechanism modeled the structure of the language more faithfully; on the other hand, it was an architecture that was easily parallelizable on modern hardware (see Section 3 for details). This led to a series of further milestones and improvements: In 2018, the *Bidirectional Encoder Representations from Transformers* (BERT) model [7] was introduced, a successor to the original transformer, which inspired a vast number of successors on its own, such as RoBERTa [22], or DistilBERT [32]. BERT (and its successors) were notable because classical pipelines (e.g., defining text representations, carrying out parts-of-speech tagging) were all subsumed by BERT-type models [36], which could easily be fine-tuned to specific tasks. At roughly the same time as the BERT model, the *Generative Pre-Trained Transformer* (GPT) model was introduced by OpenAI [28]. This was a further variation on the original transformer architecture and is the first version of the model that underlies ChatGPT, which was released in 2022 [25], and is closely related to InstructGPT [27].

The last milestone consists of the LLaMA [38] and LLaMA2 models [39] introduced in 2023, months after GPT-4 [26]. Their importance lies in being the first publicly released models, the code and weights of which were easily accessible and rivaled the performance of GPT-4; in the technical report associated with GPT-4 it is stated: "*this report contains no further details about the architecture (including model size), hardware, training compute, dataset construction, training method, or similar*". The LLaMA models led to a democratization of language models and to a large number of further successors, such as Stanford's Alpaca[3] model, or the Vicuna[4] model, which have since been used in a wide array of contexts. As these models evolved, the number of their parameters, as well as the sizes of the dataset on which they were trained, kept increasing, from the order of millions of parameters (in case of [3, 40, 7]), to billions [39, 38], to trillions [8, 30], see Figure 1. While the main trend indicates increasing model sizes, there is a countertrend to make the models smaller while retaining the performance. The DistilBERT model is an example of this. Scaling the size of the architectures and the amount of training data enabled unprecedented capabilities for the resulting LLMs, eliminating the need for fine-tuning for specific tasks.

[3] https://github.com/tatsu-lab/stanford_alpaca
[4] https://lmsys.org/blog/2023-03-30-vicuna/

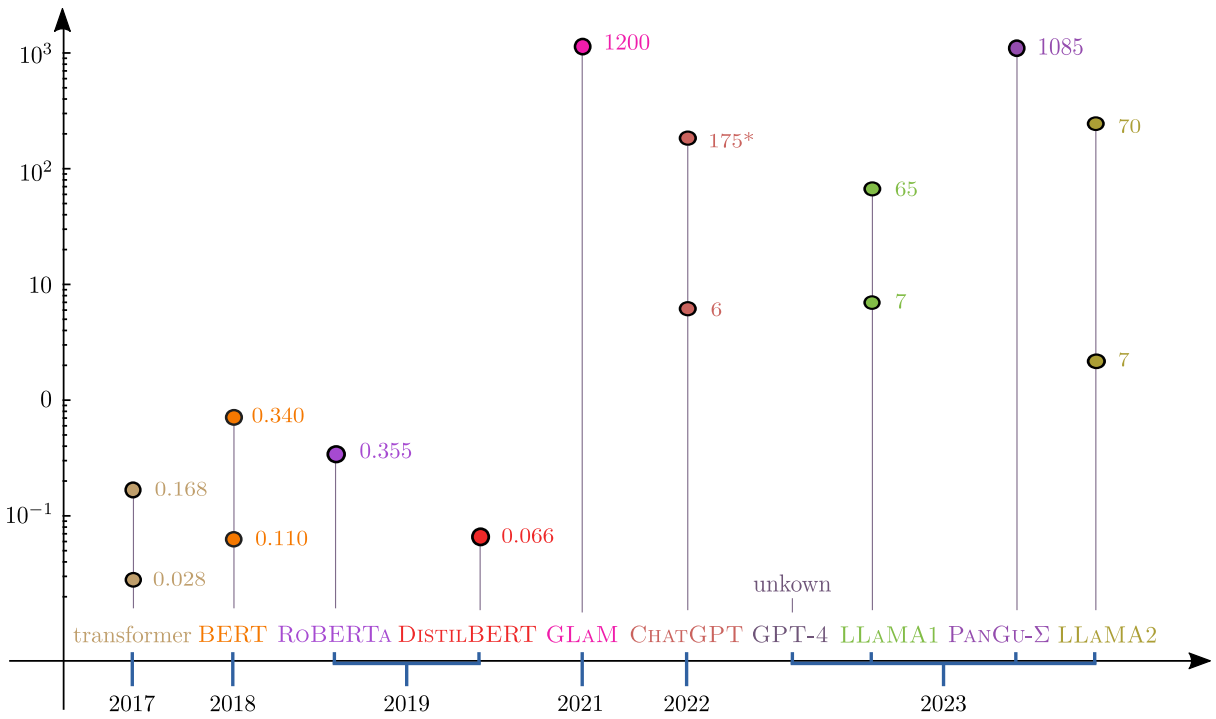

Figure 1: A selection of representative, modern language models is presented along with their parameter counts (in billions), which are displayed above. The y-axis is a log-axis, and model ranges are displayed (two horizontal dots), where available, for each model. We observe that a wide range of parameters appears, between 28 million and 1.2 trillion. For ChatGPT, exact parameter counts are not available but are taken from InstructGPT, which is a sibling model on which ChatGPT is based. For GPT-4, parameter counts are not available.

3 Technical Background

In the following, we seek to give a brief introduction to the inner workings of LLMs. We refer to [45, 23] for surveys and further details. We do not strive to present state-of-the-art models and techniques in natural language processing but focus on a conceptual understanding of the functionality of LLMs. In particular, we will restrict the presentation to one of the most popular architectures, the transformer [40] (see Section 2 for context regarding the importance of this model). Our description below is a simplified, mathematical summary loosely based on the (open-source) code[5] of GPT-2 [29], see also Figure 2 for an overview.

3.1 Transformer Architecture

Let us explore how a transformer architecture predicts text based on some provided input, often referred to as *prompt*. In line with successful models, such as the GPT and LLaMA series, we focus on the setting, where the model iteratively predicts the next word pieces (i.e., tokens) based on a given sequence of tokens. This procedure is coined *autoregressive* since the prediction of new tokens is only based on previous tokens. Such conditional sequence generation tasks using autoregressive transformers are often referred to as *decoder-only* settings. Although there is a maximum *context length* in practice, we will work with sequences of arbitrary length for ease of presentation. We define the shorthand notation $S^* := \bigcup_{n\in\mathbb{N}} S^n$ for a set S to denote the set of sequences $\boldsymbol{s} = (s^{(i)})_{i=1}^n \subset S$ with arbitrary length $n \in \mathbb{N}$. For a function $\mathcal{F}\colon S_1 \to S_2$, we denote by $\mathcal{F}^*\colon S_1^* \to S_2^*$ the *entrywise* applied mapping given by

$$\mathcal{F}^*(\boldsymbol{s}) := (\mathcal{F}(s^{(i)}))_{i=1}^n. \tag{1}$$

Tokenization $\mathcal{K}$. First, we want to clarify how we define word pieces, i.e., tokens. Mathematically speaking, we seek an injective mapping $\mathcal{K}\colon A^* \to T^*$ from the given text, i.e., a sequence $\boldsymbol{a} = (a^{(i)})_{i=1}^N$ of characters in an alphabet A to a sequence of $n \leq N$ tokens $(t^{(i)})_{i=1}^n$, where typically $T := \{1, 2, \ldots, M\}$.

[5] https://github.com/openai/gpt-2

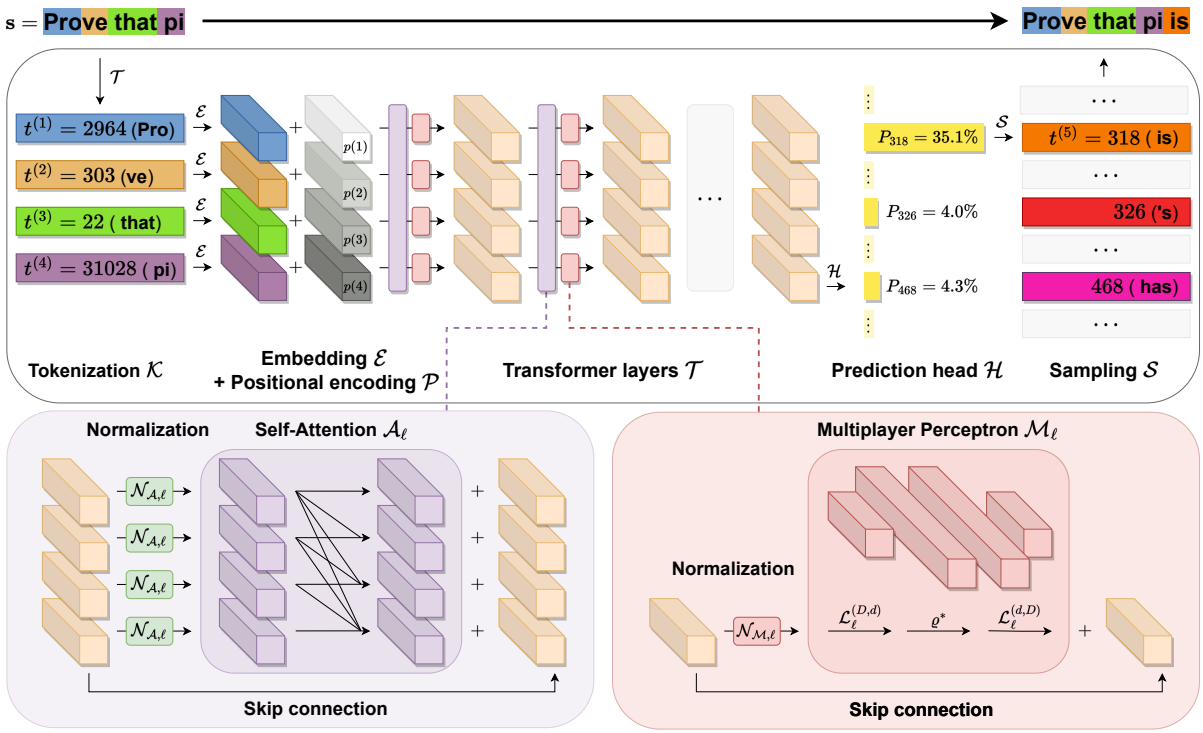

Figure 2: Illustration of the operations of an LLM for the input text "`Prove that pi`". The token indices, as well as the probabilities for the next token, are taken from GPT-2 [29] using the implementation in the `transformers` library [42]. The highest probability for the next token is assigned to `318` corresponding to the word "`is`".

To represent text on a computer, we could encode every character $a^{(i)}$ individually, similar to *Unicode*. While this would lead to a small vocabulary T of tokens, it yields long sequences where individual tokens do not capture any linguistic information. For LLMs, one often employs *subword tokenization* [34], which creates a vocabulary of subwords by analyzing large text corpora and iteratively merging frequently occurring sequences of characters. Akin to a compression problem, one balances the length n of the sequences and the size[6] M of the vocabulary. As an example, the GPT-4 tokenizer[7] splits the word "discontinuity" into the subwords "dis" (prefix), "contin" (subword capturing the root of "continuous"), and "uity" (suffix). Another example can be found in Figure 2.

Embedding $\mathcal{E}$. To use these tokens (given by the indices of the subwords in the vocabulary) in a neural network, we embed each token $t^{(i)}$ into the same Euclidean space $E := \mathbb{R}^d$. Intuitively, we seek a map $\mathcal{E}\colon T \to E$, such that the distance $\|\mathcal{E}(t^{(i)}) - \mathcal{E}(t^{(j)})\|$ corresponds to the linguistic similarity of the subwords represented by the tokens $t^{(i)}$ and $t^{(j)}$. In practice, such an embedding is often initialized with a sequence of M random initial embeddings and learned jointly with the transformer model from data.

Positional Encoding $\mathcal{P}$. Since $\mathcal{E}$ operates on each token $t^{(i)}$ independently, the embeddings $\mathcal{E}(t^{(i)})$ do not contain information on the position i of the (sub)words within a sentence[8]. Thus, one typically adds so-called *positional encodings*, which can be described by a mapping $\mathcal{P}\colon E^* \to E^*$. A commonly used choice is of the form

$$\mathcal{P}((e^{(i)})_{i=1}^n) := (e^{(i)} + p(i))_{i=1}^n, \tag{2}$$

where $p\colon \mathbb{N} \to E$ can be a prescribed injective function, e.g., a sinusoid [40], or learned (similar to the embedding $\mathcal{E}$) [28].

[6]The LLaMA and LLaMA2 models employ vocabularies T with $M = 32000$ tokens [39]. GPT-2 uses $M = 50257$, and other models in the GPT series, e.g., GPT-3.5-turbo and GPT-4, even use $M = 100277$ tokens, see https://github.com/openai/tiktoken.

[7]See https://platform.openai.com/tokenizer.

[8]In some settings, where the transformer architecture is permutation invariant, positional encodings are strictly necessary. In our decoder-only setting, this is not the case [15]; however, the encodings still seem to improve performance

In summary, tokenization $\mathcal{K}$, followed by an application of $\mathcal{E}$ to each token and the positional encoding $\mathcal{P}$, maps the text $\boldsymbol{a} \in A^*$ to a sequence of embeddings

$$\boldsymbol{e} \coloneqq (\mathcal{P} \circ \mathcal{E}^* \circ \mathcal{K})(\boldsymbol{a}) \in E^*. \tag{3}$$

where the length of $\boldsymbol{e}$ depends on $\boldsymbol{a}$ and the tokenization algorithm.

Transformer $\mathcal{T}$. The transformer can be represented as a neural network $\mathcal{T}\colon E^* \to E^*$. It is trained to map a sequence of embeddings $\boldsymbol{e}$ to another sequence of the same length containing contextual information. Based on the desired autoregressive structure, where the prediction of the next token only depends on the previous tokens, we want the i-th element of $\mathcal{T}(\boldsymbol{e})$ to contain information about all the embeddings $(e^{(j)})_{j \le i}$, however, to be independent of $(e^{(j)})_{j>i}$.

The transformer is typically defined by a composition of $L \in \mathbb{N}$ blocks, consisting of *self-attention* maps $\mathcal{A}_\ell$, entrywise applied *normalizing layers* $\mathcal{N}_{\mathcal{A},\ell}$, $\mathcal{N}_{\mathcal{M},\ell}$, and *feed-forward multiplayer perceptrons* $\mathcal{M}_\ell$, i.e.,

$$\mathcal{T} \coloneqq \left(\left(\mathrm{Id} + \mathcal{M}_L^* \circ \mathcal{N}_{\mathcal{M},L}^*\right) \circ \left(\mathrm{Id} + \mathcal{A}_L \circ \mathcal{N}_{\mathcal{A},L}^*\right)\right) \circ \cdots \circ \left(\left(\mathrm{Id} + \mathcal{M}_1^* \circ \mathcal{N}_{\mathcal{M},1}^*\right) \circ \left(\mathrm{Id} + \mathcal{A}_1 \circ \mathcal{N}_{\mathcal{A},1}^*\right)\right). \tag{4}$$

In the above, Id denotes the identity mapping, commonly known as a *skip* or *residual connection*, and the addition is understood entrywise. The indices of the layers $\mathcal{N}$, $\mathcal{M}$, and $\mathcal{A}$ in (4) indicate the use of different trainable parameters in each of the layers. Let us describe these layers in more detail below.

Layers: Normalization $\mathcal{N}$. The normalizing layer can be interpreted as a re-parametrization with a learnable mean and standard deviation to stabilize training. For instance, using *layer normalization* $\mathcal{N}\colon E \to E$, we compute

$$\mathcal{N}(e) = \frac{\mathrm{diag}(s)}{\sigma}(e - \mu) + m, \tag{5}$$

where $\mu = \frac{1}{d}\sum_{i=1}^d e_i$ and $\sigma^2 = \frac{1}{d}\sum_{i=1}^d (e_i - \mu)^2$ are the mean and variance of $e \in E$, and $s, m \in E$ are learnable parameters [1].

Layers: Multilayer Perceptrons $\mathcal{N}$. The Multilayer perception (MLP) is a standard feed-forward neural network consisting of compositions of affine mappings and nonlinear activation functions. Let us define by $\mathcal{L}^{(m,n)}\colon \mathbb{R}^m \to \mathbb{R}^n$ an affine mapping $\mathcal{L}^{(m,n)}(x) \coloneqq Wx + b$, where the *weight matrix* $W \in \mathbb{R}^{n \times m}$ and the *bias vector* $b \in \mathbb{R}^m$ are learnable. Moreover, let $\varrho\colon \mathbb{R} \to \mathbb{R}$ be an activation function, e.g., the *GELU activation function* $\varrho(x) \coloneqq x\,\Phi(x)$, where Φ is the standard Gaussian cumulative distribution function [17]. A typical MLP $\mathcal{M}\colon E \to E$ used in transformers is then given by

$$\mathcal{M} \coloneqq \mathcal{L}^{(d,D)} \circ \varrho^* \circ \mathcal{L}^{(D,d)}, \tag{6}$$

where $D \in \mathbb{N}$ with $D \ge d$.

Layers: Self-Attention $\mathcal{A}$. As can be seen in (4) and Figure 2, the self-attention layer $\mathcal{A}\colon E^* \to E^*$ is the only layer that combines embeddings of different tokens; in other words, it *attends* to other tokens. Let us denote the input to the layer by $(e^{(i)})_{i=1}^n$ and focus on the i-th output. We first compute the (normalized) inner products

$$s_j^{(i)} = \frac{1}{\sqrt{k}}\left\langle \mathcal{L}_{\mathrm{query}}^{(k,d)}(e^{(i)}), \mathcal{L}_{\mathrm{key}}^{(k,d)}(e^{(j)})\right\rangle, \quad j = 1, \ldots, i, \tag{7}$$

with given $k \in \mathbb{N}$. On a high level, we can interpret $\boldsymbol{s}^{(i)} = (s_j^{(i)})_{j=1}^i \subset \mathbb{R}$ as similarities between the embedding $\mathcal{L}_{\mathrm{query}}^{(k,d)}(e^{(i)})$ of the i-th token (i.e., the so-called *query*) and the embeddings $\mathcal{L}_{\mathrm{key}}^{(k,d)}(e^{(j)})$ of the other tokens (i.e., *keys*); to satisfy the autoregressive structure, we only consider $j \le i$. To normalize $\boldsymbol{s}^{(i)}$ to probabilities, we can further use a *softmax layer* $\mathrm{softmax}\colon \mathbb{R}^* \to \mathbb{R}^*$ given by

$$\mathrm{softmax}(\boldsymbol{s}^{(i)})_j \coloneqq \frac{\exp\left(s_j^{(i)}\right)}{\sum_{k=1}^i \exp\left(s_k^{(i)}\right)}, \quad j = 1, \ldots, i. \tag{8}$$

We can now interpret $\mathrm{softmax}(\boldsymbol{s}^{(i)})_j$ as the probability for the i-th query to "attend" to the j-th key. The self-attention layer $\mathcal{A}$ can then be defined as

$$\mathcal{A}(\boldsymbol{e})_i \coloneqq \mathcal{L}^{(k,d)} \left(\sum_{j=1}^{i} \mathrm{softmax}(\boldsymbol{s}^{(i)})_j \mathcal{L}^{(k,d)}_{\mathrm{value}}(e^{(j)}) \right), \quad i = 1, \ldots, n, \tag{9}$$

where the outputs of $\mathcal{L}^{(k,d)}_{\mathrm{value}}$ are often referred to as the *values* of the token embeddings $e^{(j)}$, and where the learnable affine layer $\mathcal{L}^{(k,d)}$ maps the weighted average of values back to $E = \mathbb{R}^d$.

Note that in practice, one typically considers a sum of $h \in \mathbb{N}$ such attention layers (so-called *heads*), each with dimension $k = d/h$ [40, 21]. Moreover, instead of considering vectors of variable length i, a mask enforces the autoregressive structure so that all operations can be efficiently batched.

Prediction Head $\mathcal{H}$. The *prediction head* or *un-embedding layer* can be represented as a mapping $\mathcal{H} \colon E^* \to \Delta^M$, where

$$\Delta^M \coloneqq \left\{ P \in [0,1]^M \colon \sum_{i=1}^{M} P_i = 1 \right\} \tag{10}$$

denotes the probability simplex in $\mathbb{R}^M$. It maps the sequence of transformed embeddings $(\tilde{e}^{(i)})_{i=1}^n \coloneqq \mathcal{T}(\boldsymbol{e})$ to a vector $P \in \Delta^M$, where P_i describes the probability of predicting $i \in T$ as the next token. Since the transformed embedding of the last token, i.e., $\tilde{e}^{(n)}$, contains information about the whole input text, a simple approach is to use a linear mapping composed with a softmax layer and define

$$P \coloneqq (\mathrm{softmax} \circ \mathcal{L}^{(M,d)})(\tilde{e}^{(n)}). \tag{11}$$

Sampling $\mathcal{S}$. There are multiple *sampling* strategies $\mathcal{S} \colon \Delta^M \to T$ to arrive at the final prediction for the next token $t^{(n+1)}$, see, e.g., [18]; the arguably simplest one, so-called *greedy sampling*, predicts the token with the highest probability, i.e.,

$$t^{(n+1)} = \mathcal{S}(P) \coloneqq \operatorname*{arg\,max}_{i=1,\ldots,M} P_i, \tag{12}$$

see Figure 2. One can then apply the same operations to the extended sequence $\boldsymbol{t} = (t^{(i)})_{i=1}^{n+1}$, i.e.,

$$t^{(n+2)} \coloneqq (\mathcal{S} \circ \mathcal{H} \circ \mathcal{T} \circ \mathcal{P} \circ \mathcal{E}^*)\,(\boldsymbol{t}) \tag{13}$$

to iteratively compute further tokens[9]. Due to the autoregressive structure, this can efficiently be done by caching the previous (intermediate) results and only considering the computations for the new token.

3.2 Training

During training, we transform text corpora into sequences of tokens, such that, for a given sequence $(t_i)_{i=1}^n$, we already know the next token t_{n+1} based on the underlying text. One can thus compute the deviation D between the predicted probabilities P of the next token and the ground-truth t_{n+1}, typically using a *cross-entropy loss*; in practice, this procedure can be parallelized to compute average losses across many predictions. Using *automatic-differentiation*, one then computes the derivative $\nabla_\theta D$ of the average loss D with respect to the learnable parameters $\theta \in \mathbb{R}^p$ of the transformer $\mathcal{T}$, the embedding $\mathcal{E}$, the prediction head $\mathcal{H}$ (and the positional encoding $\mathcal{P}$ if it is trainable). Updating the parameter by subtracting a sufficiently small multiple $\lambda \in (0, \infty)$ of the derivative, i.e., $\theta_{k+1} = \theta_k - \lambda \nabla_\theta D$, one can iteratively minimize the loss—a method known as *stochastic gradient descent*. This is the essential mechanism by which word occurrence probabilities are estimated by training from raw data. With substantial engineering efforts, more elaborate versions of such training schemes can be parallelized on large GPU clusters and scaled to immense amounts of data. To get an idea of the dimensions, the largest LLAMA2 model with $p = 70 \cdot 10^9$ parameters was trained for more than 1.7 million GPU hours on about 2 trillion tokens of data from publicly available sources [39].

[9] There is usually a stopping criterion based, e.g., on a special token or the entropy of P.

3.3 Training Costs and Emissions

Training LLMs, as described in the previous section, is a computationally very intensive process and, therefore, costly to carry out in terms of electricity usage (assuming all the hardware would be in place). However, information about training costs and CO_2 emissions is not consistently provided in the literature. Notable exceptions include the LaMDA model. The authors [37] report that a total of 451MWh was consumed during training, and, as a result, approximately 26 tons of CO_2 were emitted. Using historic US prices[10] of 0.148 dollars per kWh, this amounts to a cost of $66,748$ dollars. We note that costs may vary by country and by the energy source used to produce energy [35]. The GLaM model consumes, when trained on the largest dataset, similarly 456MWh and emits 40.2 tons of CO_2, which places it thus in a similar category to the LaMDA model in terms of cost and emission.

However, more modern LLMs incur significantly more energy consumption and emissions. For instance, the training of LLaMA2 (using 1.7 million hours on GPUs with a power consumption of about 400W) emitted more than 291 tons of Carbon dioxide equivalent (CO_2-eq) [39]. LLM vendors (such as OpenAI) typically do not release information about the costs (either in terms of consumed megawatt-hours or (rented) GPU-hours) of training their models, so only vague estimates are possible, which are nonetheless staggering. For example, training the older-generation GPT-3 model [4] was estimated, using GPU-hour prices from that time, to run up costs of approximately 4.6 million dollars [20].

4 LLMs for Mathematics

With the foundations of LLMs now well-established, we turn our attention to their application in supporting professional mathematicians. While mathematicians engage in a broad spectrum of mathematical activities, such as performing simulations, modeling, and computation, we focus on the arguably most important task: the capacity of LLMs to generate mathematical proofs. In this sense, our article differs from [41], which reviews other general deep-learning approaches to mathematics.

When using an LLM to assist in the task of theorem proving, the simplest way is to directly prompt the model to prove the statement instead of using it for individual steps or other tasks that will be described below. However, many issues have been found with this approach. A primary concern is that mathematical arguments hinge on the precision of logic; a single wrong statement very likely invalidates the entire proof. Assuming that LLMs have a non-negligible, independent probability of error with each predicted word, the *likelihood of producing a correct proof diminishes exponentially with increasing text length.* This was also empirically observed in [16], where an LLM turned out to have a higher accuracy in solving computation tasks if it was asked to skip intermediate steps.

The autoregressive nature of LLMs introduces another critical issue. Once a statement is made, LLMs typically do not revisit or revise their arguments. *This process diverges significantly from the methodologies of most mathematicians.* Rarely does a mathematician draft a complete and detailed proof in a single attempt. Instead, the process often involves crafting a rough sketch, omitting small steps in which we have confidence, iterating, and refining until the proof reaches completion.

Furthermore, LLMs may construct entirely *valid proofs for questions different from those posed.* Such an instance was exemplified in the introduction when the LLM was prompted for a function that is continuous at only one point. Given the prevalence of similar but distinct problems in training datasets, LLMs are likely to respond to the more commonly encountered variations of a question. In this case, a question for a function that is *dis*continuous at only one point. Similarly, they may prove a theorem under stronger assumptions than stated without making this explicit.

Finally, being based solely on statistical relations of language, LLMs struggle considerably with *arithmetic problems*: These often occur when an LLM has to complete a task, such as carrying out addition or multiplication (in particular, if the involved numbers are large). The reason for this is that no numerical solver is built into LLMs. Steps towards overcoming this have recently been made by employing a Toolformer approach [33]. An instance of this is the WolframAlpha plugin that is available for GPT-4.

In summary, when LLMs are used to prove theorems, they are susceptible to a range of errors. These errors were examined in [13], to be discussed in the following chapter. Consequently, a more collaborative approach, incorporating human expertise, is advisable. The following strategies appear to be sensible:

[10] https://data.bls.gov/timeseries/APU000072610

- **Literature/search engine:** The LLM can be prompted to explain a definition, find the established name for a vaguely described concept, or find references for a certain statement. In this context, two crucial considerations arise. First, LLMs are known for generating plausible yet fictitious content. This phenomenon is often referred to as *hallucinations*. Therefore, its answer to our queries needs to be verified. Second, the LLM may exacerbate biases in research. This can occur when an LLM overlooks inadequately cited work, effectively burying it while disproportionately recommending over-hyped articles.
- **Brainstorming/Idea Generation:** An LLM can be asked to provide a high-level idea of how to prove a theorem. While this will not produce the full result, it closely resembles the style of a mathematician. There is, however, no guarantee that this idea will be very insightful or lead to something. Being trained on a large corpus of mathematical arguments, an LLM will likely be biased towards recommending the most standard ideas. This may not be helpful for a mathematician who is already an expert in a specific field. However, it could be very valuable for a mathematician trying to enter a new area.
- **Proof-checking:** An LLM can be asked to find mistakes in a given proof. While there is no guarantee whatsoever that it will find all errors, the ones it finds can often be immediately confirmed as actual mistakes by a mathematician. This is helpful but not reliable. The LLM will likely focus on syntactical correctness over semantic correctness and hence overlook complex errors.
- **Collaborative writing:** An LLM can be asked to provide parts or a sketch of a proof and then, after taking feedback from the user, improve parts, repair errors, and add more details. In [5], the interactive performance of three LLMs (InstructGPT, ChatGPT, and GPT-4) on mathematical queries has been measured on a cohort of users. It was attempted to solve mathematical problems by using these models as an assistant. Asking definitions, general mathematical questions (not strictly related to the problem), and proof steps were the three most common use cases. It is important to keep in mind that this approach still is susceptible to introducing errors. The study found that self-assessment of individuals—whether they had correctly solved the problem using an LLM—was not always correct.

The approaches above are sensible for the successful employment of modern multi-purpose LLMs. In addition, we anticipate that LLMs specifically designed to prove theorems will be developed in the future. One avenue to achieve this is by combining LLM-generated proofs with interactive theorem provers [6, 2]. First steps in this direction have already been taken and appear to be very promising [12, 11, 43, 44].

5 Measuring LLM Performance on Mathematics

In [13], an empirical study was carried out to study the mathematical reasoning abilities of three LLMs which were considered to be state-of-the-art in terms of general performance: Two ChatGPT versions (9-January-2023 and 30-January-2023) and GPT-4. The prompts used to carry out the evaluation correspond to some of the use cases discussed in Section 4.

(PP) *Producing proofs:* Exercises from well-known textbooks (*Probability Theory* by R. Durret [9], *Topology* by J. R. Munkres [24], and *Functional Analysis* by W. Rudin [31]) were fed to the LLMs.

(FH) *Filling holes:* Proofs with a gap were given to the LLMs, and they were asked to fill the gap.

(SE) *Acting as a mathematical search engine:* The LLMs were asked to give a definition of concepts such as: "*What is a Banach space?*". In addition, they were asked to provide the name of definitions, as in "*How is a complete normed vector space called?*". Finally, they were prompted to provide proof ideas used in famous theorems.

(CO) *Computation:* The LLMs were given mathematical tasks in which quantities had to be computed.

To carry out this analysis, the GHOSTS dataset was introduced; see Table 1 for a more detailed description of its subdatasets (which make up the acronym "GHOSTS"), as well as how they correspond to the use cases listed above. It consists of 709 prompts, each of which was given to the three considered models. The responses of the LLMs were rated by professional mathematicians[11] on a scale between one (failure to understand the query) and five (perfect or almost perfect output). The obtained ratings are shown in Figure 3. We can make the following observations:

[11]The authors of the present paper form a subset of the evaluators.

Table 1: A summary of all the files from all the subdatasets comprising the GHOSTS dataset, together with their size, i.e., the number of prompts and their associated attribute tags.

Subdataset Name	Size	Type
***G**rad-Text*	130	PP
***H**oles-in-Proofs*	162	FH
***O**lympiad-Problem-Solving*	101	PP
***S**ymbolic-Integration*	100	CO
*MA**T**H*	138	CO
***S**earch-Engine-Aspects*	78	SE

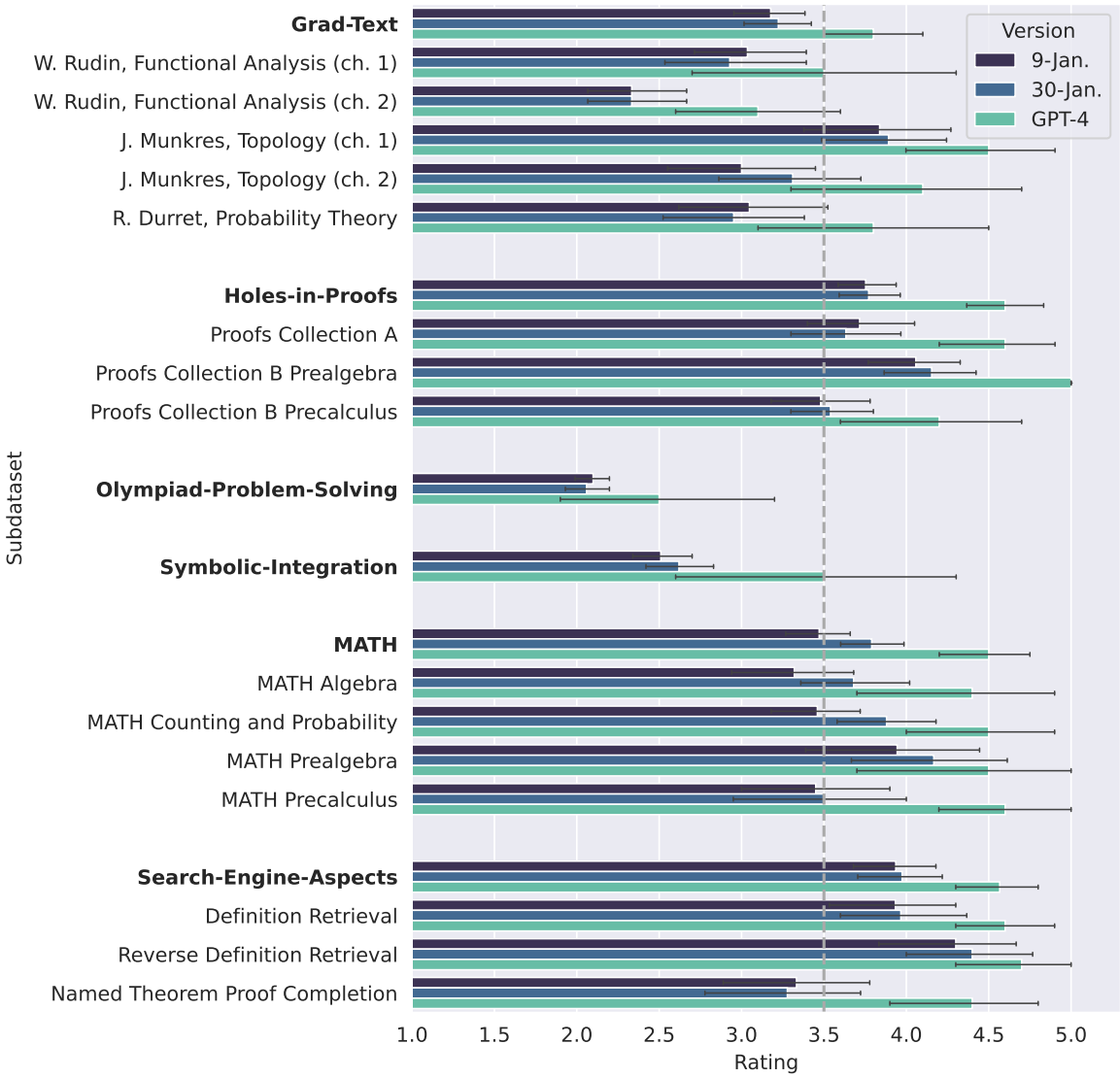

Figure 3: Average rating for each file in each subdataset (bold) of GHOSTS for the studied versions of ChatGPT and GPT-4. The maximal ranking is `5`, and the minimal ranking, where the question was at least understood, is `2`; the rating of `1` indicates that the answer completely misses the question. Thus, a reasonable passing grade, i.e., 50% of points, corresponds to a score of `3.5`, indicated by the dotted line. The error bars represent 95% confidence intervals.

Figure 4: A Sankey diagram of how the ratings evolve from the 9-January-2023 version of ChatGPT to the 30-January-2023 version ChatGPT, and subsequently to GPT-4 (from top to bottom), with all models evaluated on a representative subset of GHOSTS. Each score is color-coded, and the line widths are proportional to the number of ratings.

- *The LLMs work well as a search engine:* ChatGPT and GPT-4 achieved an excellent score when we asked for definitions of a concept or the name of a theorem or definition.
- *ChatGPT and GPT-4 struggle with hard questions:* No version of the tested LLMs achieved satisfactory results on the hardest problem set—*Olympiad-Problem-Solving*. Similarly, on the functional analysis questions from Rudin (Chapter 2)—arguably the second most advanced set of questions in the dataset—the results were underwhelming. The ratings were substantially better for more straightforward questions, such as the exercises in topology, which only ask for simple set theory and Boolean logic.
- *Good results for simple computations:* Despite not having a built-in numerical solver, GPT-4 performed reasonably well on questions requiring simple computation. For more sophisticated computation involved in *Symbolic-Integration*, ChatGPT failed and GPT-4 barely achieved a passing grade.
- *User input can have a positive effect:* On the *Holes-in-Proofs* subdataset, we see excellent results in some of the problems. It appears that the additional context given by the user helps the LLMs to produce more truthful solutions. A similar observation was made in a separate experiment where carefully crafted prompts (so-called *prompt engineering*) slightly increased the score of the LLM on the *Olympiad-Problem-Solving* subdataset [13].

The *improvement in rating, as the models become more sophisticated*, is indicated in the Sankey diagram in Figure 4, which shows how ratings change from one model version to another. We use a representative subset of GHOSTS to advise the Sankey diagram. We observe that between the 9-January-2023 version and the 30-January-2023 version, scores are approximately shuffled, and no substantial increase of the net score occurs. For the newer generation, i.e., GPT-4, we can observe a significant improvement in the ratings. This supports the general trend that more modern models also perform better on challenging mathematical reasoning tasks.

In [13], a subdivision of each question type (as indicated in Figure 3) was made, and a much more fine-grained benchmark was introduced that also differentiates potential failure modes. We refer the reader to [13] for further information on more detailed analyses.

6 Conclusion

Our study has highlighted how LLMs possess a remarkable capacity to assist mathematicians in various ways, from detecting and filling gaps in theorems to acting as search engines and finding definitions from descriptions of mathematical objects. We have shed light on the inner mechanism of the core piece of architecture that powers modern LLMs, the transformer, and how the way they produce an answer to mathematical questions differs starkly from human reasoning.

The ability of LLMs to interact with users in a natural language format has made mathematics more accessible, allowing for a broader range of individuals to engage with mathematical research and education. While the full potential of LLMs in automating mathematics is yet to be realized, our findings suggest a promising synergy between human mathematicians and artificial intelligence. High exposure of the work of mathematicians to the effects of LLMs has also been reported in [10]. However, we want to caution that, currently, LLMs are not on a trajectory to replace mathematicians. In [13], it was shown that even the best-performing model has trouble with mathematics on upper-undergraduate difficulties, such as when it is tasked to solve exercises from W. Rudin's *Functional Analysis* [31]. The performance of LLMs has been reported to be below that of humans also in related domains, such as coding challenges in computer science [19]. Nonetheless, we anticipate that the emergence of LLMs will be a challenge for education and research. Simple exercises or homework and individual steps in mathematical research will be gradually supported by automation or become obsolete.

Acknowledgements

S. Frieder and T. Lukasiewicz were partially supported by the AXA Research Fund.

References

[1] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] B. Barras, S. Boutin, C. Cornes, J. Courant, J.-C. Filliatre, E. Gimenez, H. Herbelin, G. Huet, C. Munoz, C. Murthy, et al. *The Coq proof assistant reference manual: Version 6.1*. PhD thesis, Inria, 1997.

[3] Y. Bengio, R. Ducharme, and P. Vincent. A neural probabilistic language model. *Advances in Neural Information Processing Systems*, 13, 2000.

[4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[5] K. M. Collins, A. Q. Jiang, S. Frieder, L. Wong, M. Zilka, U. Bhatt, T. Lukasiewicz, Y. Wu, J. B. Tenenbaum, W. Hart, et al. Evaluating language models for mathematics through interactions. *arXiv preprint arXiv:2306.01694*, 2023.

[6] L. de Moura, S. Kong, J. Avigad, F. Van Doorn, and J. von Raumer. The Lean theorem prover (system description). In *Automated Deduction-CADE-25: 25th International Conference on Automated Deduction*, pages 378–388, 2015.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[8] N. Du, Y. Huang, A. M. Dai, S. Tong, D. Lepikhin, Y. Xu, M. Krikun, Y. Zhou, A. W. Yu, O. Firat, et al. GLaM: Efficient scaling of language models with mixture-of-experts. In *International Conference on Machine Learning*, pages 5547–5569. PMLR, 2022.

[9] R. Durrett. *Probability: Theory and Examples*. Cambridge Series in Statistical and Probabilistic Mathematics. Cambridge University Press, 2019.

[10] T. Eloundou, S. Manning, P. Mishkin, and D. Rock. GPTs are GPTs: An early look at the labor market impact potential of large language models. *arXiv preprint arXiv:2303.10130*, 2023.

[11] E. First, M. N. Rabe, T. Ringer, and Y. Brun. Baldur: whole-proof generation and repair with large language models. *arXiv preprint arXiv:2303.04910*, 2023.

[12] S. Frieder, M. Alawadhi, Trimmel, Rashid, and K. Gy. LLM vs ITP. In *The 3rd Workshop on Mathematical Reasoning and AI at NeurIPS'23*, 2023.

[13] S. Frieder, L. Pinchetti, R.-R. Griffiths, T. Salvatori, T. Lukasiewicz, P. C. Petersen, A. Chevalier, and J. Berner. Mathematical capabilities of ChatGPT. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

[14] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, et al. The Pile: An 800GB dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

[15] A. Haviv, O. Ram, O. Press, P. Izsak, and O. Levy. Transformer language models without positional encodings still learn positional information. *arXiv preprint arXiv:2203.16634*, 2022.

[16] D. Hendrycks, C. Burns, S. Kadavath, A. Arora, S. Basart, E. Tang, D. Song, and J. Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[17] D. Hendrycks and K. Gimpel. Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*, 2016.

[18] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019.

[19] A. Koubaa, B. Qureshi, A. Ammar, Z. Khan, W. Boulila, and L. Ghouti. Humans are still better than ChatGPT: Case of the IEEEXtreme competition. *arXiv preprint arXiv:2305.06934*, 2023.

[20] C. Li. OpenAI's GPT-3 language model: A technical overview, 2020. https://lambdalabs.com/blog/demystifying-gpt-3.

[21] L. Liu, J. Liu, and J. Han. Multi-head or single-head? an empirical comparison for transformer training. *arXiv preprint arXiv:2106.09650*, 2021.

[22] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*, 2019.

[23] B. Min, H. Ross, E. Sulem, A. P. B. Veyseh, T. H. Nguyen, O. Sainz, E. Agirre, I. Heintz, and D. Roth. Recent advances in natural language processing via large pre-trained language models: A survey. *ACM Computing Surveys*, 56(2):1–40, 2023.

[24] J. R. Munkres. *Topology*. Prentice-Hall, 2000.

[25] OpenAI. Introducing ChatGPT, 2022. https://openai.com/blog/chatgpt.

[26] OpenAI. GPT-4 technical report. *arXiv preprint 2303.0877*, 2023.

[27] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

[28] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, et al. Improving language understanding by generative pre-training, 2018. https://openai.com/research/language-unsupervised.

[29] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners, 2019. https://github.com/openai/gpt-2.

[30] X. Ren, P. Zhou, X. Meng, X. Huang, Y. Wang, W. Wang, P. Li, X. Zhang, A. Podolskiy, G. Arshinov, et al. PanGu-Σ: Towards trillion parameter language model with sparse heterogeneous computing. *arXiv preprint arXiv:2303.10845*, 2023.

[31] W. Rudin. *Functional analysis*. McgGraw-Hill, 1991.

[32] V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

[33] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

[34] R. Sennrich, B. Haddow, and A. Birch. Neural machine translation of rare words with subword units. *arXiv preprint arXiv:1508.07909*, 2015.

[35] E. Strubell, A. Ganesh, and A. McCallum. Energy and policy considerations for deep learning in NLP. *arXiv preprint arXiv:1906.02243*, 2019.

[36] I. Tenney, D. Das, and E. Pavlick. BERT rediscovers the classical NLP pipeline. *arXiv preprint arXiv:1905.05950*, 2019.

[37] R. Thoppilan, D. De Freitas, J. Hall, N. Shazeer, A. Kulshreshtha, H.-T. Cheng, A. Jin, T. Bos, L. Baker, Y. Du, et al. LaMDA: Language models for dialog applications. *arXiv preprint arXiv:2201.08239*, 2022.

[38] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[39] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 2017.

[41] G. Williamson. Is deep learning a useful tool for the pure mathematician? *arXiv preprint arXiv:2304.12602*, 2023.

[42] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, 2020.

[43] K. Yang and J. Deng. Learning to prove theorems via interacting with proof assistants. In *International Conference on Machine Learning*, pages 6984–6994. PMLR, 2019.

[44] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger, and A. Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. *arXiv preprint arXiv:2306.15626*, 2023.

[45] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 2023.